# Machine Coding: News Feed with Infinite Scrolling

- **Problem:** Build a news feed (like Facebook / LinkedIn / X) that loads the next page of posts automatically as the user scrolls down.
- **Tech stack:** React (Vite, JavaScript) *(default, not specified)*
- **Time limit:** 60 minutes *(default, not specified)*

---

## PART A: Thought process

Every machine coding task follows the same loop. Say each step out loud before you do it.

1. **Clarify requirements** – functional, non-functional, and what is out of scope.
2. **List interactions** – every user action and its expected result (this becomes the checklist).
3. **Sketch the architecture** – components, hooks, services and who talks to whom.
4. **Design state** – the minimum state, who owns it, what is derived.
5. **Define data and contracts** – sample data and the API shape (mock it if there is no API).
6. **Trace one flow end to end** – input → state → side effect → UI.
7. **Find the hard parts** – solve them on paper before you code.
8. **Build in layers** – simplest working version first, then one working layer at a time.
9. **Cover states and edge cases** – loading, empty, error, end, races.
10. **Wrap up** – trade-offs and what you would add with more time.

### Time budget (60 minutes)

| Minutes | Activity |
|---|---|
| 0 – 8 | Requirements, clarifying questions, assumptions |
| 8 – 15 | Architecture, state, API contract (mock) |
| 15 – 40 | Build: mock API → reducer/hook → list → sentinel/observer |
| 40 – 50 | Loading skeleton, error + retry, empty and end states |
| 50 – 55 | Accessibility, memo, quick manual test of every state |
| 55 – 60 | Walk the interviewer through trade-offs and next steps |

> **Tip: think out loud.** The interviewer grades how you reason, not only the final code. Say "I am choosing a cursor instead of a page number because…" before you type it.

---

# PART B: Apply it to this problem

## 1. Understand the requirements

### Functional requirements

- Show a list of posts (author, time, title, body, likes), newest first.
- Load the first page on mount.
- When the user scrolls near the bottom, load the next page automatically and append it.
- Show a loading indicator (skeleton) while a page is loading.
- If a page fails, show an error with a **Retry** button; do not lose posts already shown.
- When there are no more posts, show "You're all caught up".
- If there are no posts at all, show an empty state.

### Non-functional requirements

| Area | What it means here |
|---|---|
| Performance | Never fire two requests for the same page; old cards must not re-render when a page is appended; trigger the fetch *before* the user hits the bottom. |
| Scalability | Cursor-based pagination so new posts at the top don't shift pages; architecture ready for virtualization when the list gets huge. |
| Error handling | Failed page → error UI + manual retry, no infinite auto-retry loop; aborted requests are ignored silently. |
| Accessibility | `role="feed"` + `aria-busy`, each post an `<article>` with `aria-posinset`, focusable cards, `role="alert"` for errors, reduced-motion for the skeleton. |
| Reusability | Pagination logic lives in a `useInfiniteFeed(fetchPage)` hook; visibility logic in a generic `useOnScreen` hook. Both work for any list. |
| Extensibility | Swapping the mock for a real API means changing one function; adding pull-to-refresh or "new posts" is a new reducer action. |

### Clarifying questions to ask

1. Is the API **offset/page-based** or **cursor-based**? Can new posts arrive while I scroll?
2. Is there a total count, or does the API only tell me "has more"?
3. Should scrolling be automatic only, or is a "Load more" button also required (accessibility / footer reachability)?
4. How large can the feed get in one session — hundreds or tens of thousands of posts (do we need virtualization)?
5. Do posts have images/videos (lazy loading, layout shift)?
6. Should the scroll position be restored when the user navigates back to the feed?

### Assumptions we will build with

- No real backend: a **mock API** `fetchPosts({ cursor, limit, signal })` with ~800 ms latency and a 15% random failure rate.
- **Cursor-based** pagination: `nextCursor` is the id of the last post; `null` means no more pages.
- Page size = 10; total mock posts = 57 (so the last page is partial and the end state is reachable).
- Automatic loading via **IntersectionObserver**; manual button only for Retry.
- Text-only posts, no virtualization (mentioned as a follow-up).

---

## 2. Identify interactions / use cases

| Action / input | Expected behavior / output |
|---|---|
| Page opens | 3 skeleton cards, first 10 posts load, skeleton disappears |
| Scroll to ~300px from the bottom | 1 skeleton card appears at the bottom, next 10 posts are appended |
| Scroll fast / sentinel fires repeatedly | Only **one** request is in flight; no duplicate posts |
| Screen is taller than the first page | Next page loads automatically until the screen is filled |
| Request fails | Error box with message + Retry; existing posts stay; no automatic re-fetch loop |
| Click Retry | Error disappears, skeleton shows, the same page is requested again |
| Last page loaded | "You're all caught up 🎉", no more requests |
| API returns 0 posts | "No posts yet. Check back later." |
| Leave the page while loading | Request is aborted; no state update on an unmounted component |
| Keyboard user tabs | Each post card receives focus with a visible outline |

This table is our implementation and test checklist.

---

## 3. Design the architecture

| Module | Responsibility |
|---|---|
| `api/feedApi.js` | Mock backend: `fetchPosts({ cursor, limit, signal })` → `{ items, nextCursor }` |
| `hooks/feedReducer.js` | Pure state transitions: start, success (append + dedupe), error |
| `hooks/useInfiniteFeed.js` | Owns feed state; exposes `loadMore`; guards duplicate requests; aborts on unmount |
| `hooks/useOnScreen.js` | Generic IntersectionObserver hook: "call me when this element is near the viewport" |
| `components/Feed.jsx` | Wires the hook + observer; renders list, skeleton, error, empty, end |
| `components/PostCard.jsx` | One post (memoized) |

| Module | Responsibility |
| --- | --- |
| `components/PostSkeleton.jsx` | Shimmer placeholder card(s) |
| `App.jsx` | Page shell (sticky header + feed) |

```
App
└── Feed  ── useInfiniteFeed(fetchPosts) ── feedReducer
    │     └── useOnScreen(sentinelRef, loadMore)
    ├── PostCard × N        (memo)
    ├── PostSkeleton        (while loading)
    ├── Error + Retry       (on error)
    ├── Empty / End message
    └── <div sentinel/>     (observed element at the bottom)
```

Start simple: the first version can be one `Feed` component with `useState` and a "Load more" button. Split out the hook and the observer once the behavior works.

---

## 4. Design the state / data model

| State or field | Purpose |
| --- | --- |
| `items` | Posts loaded so far, in display order |
| `cursor` | Cursor to send for the **next** request ( `null` = start from newest) |
| `hasMore` | `false` once the API returns `nextCursor: null` |
| `status` | `'idle' \| 'loading' \| 'error'` – a tiny state machine |
| `error` | Error message for the error UI |
| `inFlight` (ref) | Synchronous lock so two observer callbacks can't start two requests |
| `controllerRef` (ref) | `AbortController` of the pending request (cancel on unmount) |

```
export const initialFeedState = {
  items: [],
  cursor: null,
  hasMore: true,
  status: 'idle', // 'idle' | 'loading' | 'error'
  error: null,
};
```

**Senior note**

- **Derived, not stored:** `isEmpty = !hasMore && items.length === 0`, `isInitialLoad = status === 'loading' && items.length === 0`, `observerEnabled = hasMore && status === 'idle'`. Storing them would create states that can disagree.

- **One `status` instead of `isLoading` + `isError` booleans** – impossible combinations (loading *and* error) cannot exist.
- **Ownership:** the hook owns all feed state; `Feed` only reads it. `PostCard` owns nothing. The `inFlight` lock is a **ref**, not state, because it must be readable synchronously and must not trigger renders.

---

## 5. Design the data structures and contracts

### Example post

```
{
  "id": "57",
  "author": "Ravi",
  "title": "Cricket final tonight (#57)",
  "body": "Lorem ipsum dolor sit amet…",
  "createdAt": "2026-11-01T00:00:00.000Z",
  "likes": 109
}
```

The **stable `id`** is used as the React `key` (no index keys – cards would be re-mounted and lose focus), for de-duplication, and as the pagination cursor.

### API contract

```
GET /posts?cursor=<lastSeenId>&limit=10

200 → {
  "items": [ Post, … ],        // up to `limit` posts older than cursor, newest first
  "nextCursor": "47" | null    // null = no more pages
}
5xx / network error → rejected promise with an Error
```

```
fetchPosts({ cursor: null, limit: 10 })   // → { items: [#57 … #48], nextCursor: '48' }
fetchPosts({ cursor: '48', limit: 10 })   // → { items: [#47 … #38], nextCursor: '38' }
fetchPosts({ cursor: '8',  limit: 10 })   // → { items: [#7 … #1],  nextCursor: null }
```

**Why cursor and not `?page=3`?** With offsets, if 2 new posts are published while you scroll, page 3 starts 2 items earlier, so you see 2 duplicates (or skip posts if some are deleted). A cursor says "posts older than X", which is stable.

The mock also reads query params so every state can be tested: `?fail=1` (always fail), `?total=0` (empty), `?latency=2000` (slow network).

---

## 6. Trace the data flow

Key interaction: **user scrolls to the bottom and the next page is appended.**

```
User scrolls; sentinel <div> enters viewport + 300px rootMargin
        │
        ▼
IntersectionObserver callback → callbackRef.current()  (= latest loadMore)
        │
        ▼
loadMore: inFlight.current? → yes: return     (duplicate guard)
          hasMore false?    → return
          inFlight = true; new AbortController
        │
        ▼
dispatch FETCH_START → status 'loading'
        │   → observer disabled (enabled = hasMore && status==='idle')
        │   → aria-busy=true, 1 skeleton card rendered
        ▼
await fetchPosts({ cursor, limit: 10, signal })
        │
        ├── success → dispatch FETCH_SUCCESS
        │       items = [...items, ...newUnique], cursor = nextCursor,
        │       hasMore = nextCursor !== null, status 'idle'
        │       → new cards render (old cards skipped via memo)
        │       → observer re-created; if sentinel still visible it fires again
        │
        └── failure → dispatch FETCH_ERROR → status 'error'
                → error box + Retry, observer stays disabled
        ▼
finally: inFlight = false
```

---

## 7. Important technical challenges

### Challenge 1: Duplicate requests for the same page

**Problem:** The observer can fire several times in a row (fast scroll, layout shift, StrictMode double effects). `status` from state is stale inside the same tick, so two calls both see `status === 'idle'` and both fetch page 2 → duplicate posts.

**Solution:** A **synchronous ref lock** ( `inFlight` ) checked and set before any `await` , plus disabling the observer while loading, plus de-duplication by id in the reducer as a last line of defense.

```
const loadMore = useCallback(async () => {
  if (inFlight.current || !state.hasMore) return;
```

```
  inFlight.current = true;
  try { /* fetch + dispatch */ } finally { inFlight.current = false; }
}, [/* … */]);

// reducer
const seen = new Set(state.items.map((p) => p.id));
const fresh = action.items.filter((p) => !seen.has(p.id));
```

### Challenge 2: Stale closure inside the observer

**Problem:** The observer is created once with a callback. That callback captured `loadMore` from the first render, whose `cursor` is `null` forever → every scroll re-fetches page 1.

**Solution:** Keep the latest callback in a ref and let the observer call `callbackRef.current()`. The observer itself is only re-created when `enabled` changes.

```
const callbackRef = useRef(onVisible);
useEffect(() => { callbackRef.current = onVisible; });

const observer = new IntersectionObserver(([entry]) => {
  if (entry.isIntersecting) callbackRef.current();
}, { rootMargin });
```

### Challenge 3: Sentinel is still visible after a page loads

**Problem:** On a tall screen, 10 posts may not fill the viewport. IntersectionObserver only fires on **changes** of intersection; the sentinel never left the viewport, so no new event → the feed stalls with no scrollbar.

**Solution:** Disable the observer while loading (`enabled = hasMore && status === 'idle'`). When loading ends, `enabled` flips back to `true`, the effect creates a **new** observer, and a new observer always reports the current state once → if the sentinel is still visible, the next page loads. This repeats until the screen is full.

```
useOnScreen(sentinelRef, loadMore, { enabled: hasMore && status === 'idle' });
```

### Challenge 4: Errors without an infinite retry loop, and races on unmount

**Problem:** If the observer stays active on error, the visible sentinel triggers `loadMore` → fails → triggers again… hammering a broken server. Separately, if the user leaves while a request is pending, the response would update an unmounted component.

**Solution:** `status === 'error'` disables the observer; only the user's **Retry** click calls `loadMore`. Each request gets an `AbortController`; unmount aborts it and `AbortError` is ignored.

```
useEffect(() => () => controllerRef.current?.abort(), []);
// …
catch (err) {
```

```
  if (err.name !== 'AbortError') dispatch({ type: 'FETCH_ERROR', error: err.message });
}
```

## 8. Implement in the correct order

| # | Step | Working result after the step |
|---|---|---|
| 1 | Mock API `fetchPosts` with cursor, latency, failures | Can be called from the console |
| 2 | `Feed` with `useState` + `useEffect` loading page 1 | First 10 posts on screen |
| 3 | `PostCard` with stable `key={post.id}` | Proper card UI |
| 4 | "Load more" button that appends the next page using `nextCursor` | Pagination works manually |
| 5 | Move state into `feedReducer` + `useInfiniteFeed` hook with `status` | Clean state machine, reusable hook |
| 6 | Add `inFlight` ref lock + reducer dedupe | Double clicks never duplicate |
| 7 | `useOnScreen` with IntersectionObserver + sentinel; replace the button | Infinite scroll works |
| 8 | Callback ref (stale closure) + `enabled` toggle (re-observe) | Correct cursor; tall screens fill up |
| 9 | Skeleton (initial 3, then 1), error + Retry, empty, end states | Every state visible |
| 10 | `AbortController` on unmount | No updates after unmount |
| 11 | Accessibility: `role="feed"`, `aria-busy`, `article`, focus outline, reduced motion | Screen-reader/keyboard friendly |
| 12 | `memo(PostCard)` + quick test of every row of the use-case table | Done |

## 9. Cover all states and edge cases

| State / edge case | What the user sees / how it is handled |
|---|---|
| Initial (before first response) | 3 skeleton cards, `aria-busy="true"` |
| Loading next page | Existing posts + 1 skeleton card at the bottom |
| Success | New posts appended, skeleton removed |
| Empty (0 posts) | "No posts yet. Check back later." |
| Error on first page | Error box + Retry, no posts |
| Error on a later page | Existing posts stay; error box + Retry at the bottom |
| End of feed | "You're all caught up 🎉"; observer stays off |
| Last page is partial (7 of 10) | `nextCursor: null` → end state, no extra empty request |
| Rapid scrolling / repeated observer events | `inFlight` lock → one request at a time |

| State / edge case | What the user sees / how it is handled |
|---|---|
| Overlapping page from the server | Reducer dedupes by id |
| First page doesn't fill a tall screen | Observer re-created after each load → auto-loads more |
| Unmount during a request | Request aborted, `AbortError` ignored |
| StrictMode double mount | Lock + dedupe → still exactly one copy of each post |
| Slow network | Skeleton stays; prefetch margin (300px) hides most latency |
| Reduced motion preference | Shimmer animation disabled |

## 10. Final structure

```
_47_news_feed_infinite_scrolling/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── index.css
    ├── api/
    │   └── feedApi.js            # mock backend (cursor pagination)
    ├── hooks/
    │   ├── feedReducer.js        # pure state transitions
    │   ├── useInfiniteFeed.js    # pagination + lock + abort
    │   └── useOnScreen.js        # IntersectionObserver hook
    └── components/
        ├── Feed.jsx
        ├── PostCard.jsx
        └── PostSkeleton.jsx
```

## 11. Full working code

`src/api/feedApi.js`

```
// Mock backend. Query params let you test states: ?fail=1 (always fail), ?total=0 (empty), ?
latency=2000
const params = new URLSearchParams(globalThis.location?.search ?? '');
const TOTAL = Number(params.get('total') ?? 57);
const FAIL_RATE = Number(params.get('fail') ?? 0.15);
const LATENCY = Number(params.get('latency') ?? 800);
```

```
const AUTHORS = ['Asha', 'Ravi', 'Meera', 'Kabir', 'Zoya'];
const TOPICS = ['React 20 released', 'Monsoon update', 'Markets close higher', 'New metro line opens',
'Cricket final tonight'];

// Newest first: ids TOTAL..1, so a cursor "id" means "give me posts older than this one"
const ALL_POSTS = Array.from({ length: TOTAL }, (_, i) => {
  const id = TOTAL - i;
  return {
    id: String(id),
    author: AUTHORS[id % AUTHORS.length],
    title: `${TOPICS[id % TOPICS.length]} (#${id})`,
    body: 'Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.',
    createdAt: new Date(Date.UTC(2026, 10, 1) - i * 3_600_000).toISOString(),
    likes: (id * 37) % 500,
  };
});

/**
 * GET /posts?cursor=<lastId>&limit=10
 * -> { items: Post[], nextCursor: string | null }   (null = no more pages)
 */
export function fetchPosts({ cursor = null, limit = 10, signal } = {}) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => {
      if (Math.random() < FAIL_RATE) return reject(new Error('Could not load posts. Check your
connection.'));
      const older = cursor === null ? ALL_POSTS : ALL_POSTS.filter((p) => Number(p.id) <
Number(cursor));
      const items = older.slice(0, limit);
      resolve({ items, nextCursor: older.length > limit ? items.at(-1).id : null });
    }, LATENCY);

    signal?.addEventListener('abort', () => {
      clearTimeout(timer);
      reject(new DOMException('Aborted', 'AbortError'));
    });
  });
}
```

`src/hooks/feedReducer.js`

```
export const initialFeedState = {
  items: [],
  cursor: null, // null = start from the newest post
  hasMore: true,
  status: 'idle', // 'idle' | 'loading' | 'error'
```

```
  error: null,
};

export function feedReducer(state, action) {
  switch (action.type) {
    case 'FETCH_START':
      return { ...state, status: 'loading', error: null };
    case 'FETCH_SUCCESS': {
      // Guard against duplicates if the backend ever returns an overlapping page
      const seen = new Set(state.items.map((p) => p.id));
      const fresh = action.items.filter((p) => !seen.has(p.id));
      return {
        ...state,
        items: [...state.items, ...fresh],
        cursor: action.nextCursor,
        hasMore: action.nextCursor !== null,
        status: 'idle',
      };
    }
    case 'FETCH_ERROR':
      return { ...state, status: 'error', error: action.error };
    default:
      return state;
  }
}
```

**src/hooks/useInfiniteFeed.js**

```
import { useCallback, useEffect, useReducer, useRef } from 'react';
import { feedReducer, initialFeedState } from './feedReducer';

export function useInfiniteFeed(fetchPage, pageSize = 10) {
  const [state, dispatch] = useReducer(feedReducer, initialFeedState);
  const inFlight = useRef(false); // sync lock: state updates are async, a ref is not
  const controllerRef = useRef(null);

  const loadMore = useCallback(async () => {
    if (inFlight.current || !state.hasMore) return;
    inFlight.current = true;
    const controller = new AbortController();
    controllerRef.current = controller;
    dispatch({ type: 'FETCH_START' });

    try {
      const { items, nextCursor } = await fetchPage({
        cursor: state.cursor,
        limit: pageSize,
```

```
        signal: controller.signal,
      });
      dispatch({ type: 'FETCH_SUCCESS', items, nextCursor });
    } catch (err) {
      if (err.name !== 'AbortError') dispatch({ type: 'FETCH_ERROR', error: err.message });
    } finally {
      inFlight.current = false;
    }
  }, [fetchPage, pageSize, state.cursor, state.hasMore]);

  // Cancel the pending request when the feed unmounts
  useEffect(() => () => controllerRef.current?.abort(), []);

  return { ...state, loadMore };
}
```

`src/hooks/useOnScreen.js`

```
import { useEffect, useRef } from 'react';

/**
 * Calls onVisible when targetRef's element comes within rootMargin of the viewport.
 * Toggling `enabled` off -> on creates a fresh observer, which reports the current
 * intersection immediately, so a sentinel that is STILL visible after a page loads fires again.
 */
export function useOnScreen(targetRef, onVisible, { enabled = true, rootMargin = '300px' } = {}) {
  const callbackRef = useRef(onVisible);
  useEffect(() => {
    callbackRef.current = onVisible; // always call the latest loadMore (no stale cursor)
  });

  useEffect(() => {
    const node = targetRef.current;
    if (!enabled || !node) return;
    const observer = new IntersectionObserver(
      ([entry]) => {
        if (entry.isIntersecting) callbackRef.current();
      },
      { rootMargin },
    );
    observer.observe(node);
    return () => observer.disconnect();
  }, [targetRef, enabled, rootMargin]);
}
```

`src/components/Feed.jsx`

```
import { useRef } from 'react';
import { fetchPosts } from '../api/feedApi';
import { useInfiniteFeed } from '../hooks/useInfiniteFeed';
import { useOnScreen } from '../hooks/useOnScreen';
import PostCard from './PostCard';
import PostSkeleton from './PostSkeleton';

const PAGE_SIZE = 10;

export default function Feed() {
  const { items, hasMore, status, error, loadMore } = useInfiniteFeed(fetchPosts, PAGE_SIZE);
  const sentinelRef = useRef(null);

  // Off while loading or failed: no duplicate fetches, no retry loop on error
  useOnScreen(sentinelRef, loadMore, { enabled: hasMore && status === 'idle' });

  const isEmpty = !hasMore && items.length === 0;

  return (
    <main className="feed">
      <div role="feed" aria-busy={status === 'loading'} aria-label="News feed">
        {items.map((post, i) => (
          <PostCard key={post.id} post={post} index={i} />
        ))}
      </div>

      {status === 'loading' && <PostSkeleton count={items.length === 0 ? 3 : 1} />}

      {status === 'error' && (
        <div role="alert" className="feed__error">
          <p>{error}</p>
          <button type="button" onClick={loadMore}>Retry</button>
        </div>
      )}

      {isEmpty && <p className="feed__status">No posts yet. Check back later.</p>}
      {!hasMore && items.length > 0 && <p className="feed__status">You're all caught up 🎉</p>}

      <div ref={sentinelRef} className="feed__sentinel" aria-hidden="true" />
    </main>
  );
}
```

`src/components/PostCard.jsx`

```
import { memo } from 'react';

function PostCard({ post, index }) {
  const titleId = `post-${post.id}-title`;
  return (
    <article className="post" aria-posinset={index + 1} aria-setsize={-1} aria-labelledby={titleId}
tabIndex={0}>
      <header className="post__header">
        <span className="post__avatar" aria-hidden="true">{post.author[0]}</span>
        <div>
          <strong>{post.author}</strong>
          <time dateTime={post.createdAt}>
            {new Date(post.createdAt).toLocaleString(undefined, { dateStyle: 'medium', timeStyle:
'short' })}
          </time>
        </div>
      </header>
      <h2 id={titleId}>{post.title}</h2>
      <p>{post.body}</p>
      <footer className="post__footer">♥ {post.likes}</footer>
    </article>
  );
}

// memo: appending a page must not re-render the cards already on screen
export default memo(PostCard);
```

`src/components/PostSkeleton.jsx`

```
export default function PostSkeleton({ count = 1 }) {
  return Array.from({ length: count }, (_, i) => (
    <div key={i} className="post post--skeleton" aria-hidden="true">
      <div className="post__header">
        <span className="skeleton skeleton--avatar" />
        <span className="skeleton skeleton--line" style={{ width: '40%' }} />
      </div>
      <span className="skeleton skeleton--line" style={{ width: '70%' }} />
      <span className="skeleton skeleton--line" />
      <span className="skeleton skeleton--line" style={{ width: '85%' }} />
    </div>
  ));
}
```

`src/App.jsx`

```
import Feed from './components/Feed';

export default function App() {
  return (
    <>
      <header className="app-header">
        <h1>News Feed</h1>
      </header>
      <Feed />
    </>
  );
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

`src/index.css`

```
*,
*::before,
*::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: #f0f2f5;
  color: #1c1e21;
}

.app-header {
  position: sticky;
  top: 0;
```

```
  z-index: 1;
  background: #fff;
  border-bottom: 1px solid #ddd;
  padding: 12px 16px;
}

.app-header h1 {
  margin: 0;
  font-size: 1.25rem;
}

.feed {
  max-width: 600px;
  margin: 0 auto;
  padding: 16px;
}

.post {
  background: #fff;
  border-radius: 8px;
  padding: 16px;
  margin-bottom: 12px;
  box-shadow: 0 1px 2px rgb(0 0 0 / 0.1);
}

.post:focus-visible {
  outline: 2px solid #1877f2;
}

.post__header {
  display: flex;
  gap: 10px;
  align-items: center;
}

.post__header time {
  display: block;
  font-size: 0.8rem;
  color: #65676b;
}

.post__avatar {
  display: grid;
  place-items: center;
  width: 40px;
  height: 40px;
  border-radius: 50%;
```

```
  background: #1877f2;
  color: #fff;
  font-weight: 700;
}

.post h2 {
  font-size: 1.05rem;
  margin: 12px 0 6px;
}

.post p {
  margin: 0;
  line-height: 1.5;
}

.post__footer {
  margin-top: 10px;
  color: #65676b;
  font-size: 0.9rem;
}

.skeleton {
  display: block;
  height: 12px;
  margin-top: 10px;
  border-radius: 4px;
  background: linear-gradient(90deg, #e4e6eb 25%, #f2f3f5 50%, #e4e6eb 75%);
  background-size: 200% 100%;
  animation: shimmer 1.2s infinite;
}

.skeleton--avatar {
  width: 40px;
  height: 40px;
  margin: 0;
  border-radius: 50%;
}

.post__header .skeleton--line {
  margin: 0;
}

@keyframes shimmer {
  to {
    background-position: -200% 0;
  }
}
```

```
@media (prefers-reduced-motion: reduce) {
  .skeleton {
    animation: none;
  }
}

.feed__error {
  text-align: center;
  background: #fff3f3;
  border: 1px solid #f5c2c2;
  border-radius: 8px;
  padding: 12px;
}

.feed__error button {
  padding: 6px 16px;
  border: 0;
  border-radius: 6px;
  background: #1877f2;
  color: #fff;
  cursor: pointer;
}

.feed__status {
  text-align: center;
  color: #65676b;
}

.feed__sentinel {
  height: 1px;
}
```

---

## 12. Wrap-up

### Edge cases handled

- Duplicate requests (ref lock), duplicate posts (dedupe by id), stale cursor (callback ref).
- Tall screens that don't fill on the first page (observer re-created after each load).
- Error on any page with manual Retry and no retry loop; posts never lost.
- Empty feed, partial last page, end of feed.
- Unmount during a request (AbortController), StrictMode double effects.

### Trade-offs to mention

| Decision | Trade-off |
|---|---|
| IntersectionObserver over `scroll` listener | No throttling, runs off the main thread, but needs a sentinel element and a polyfill for very old browsers. |
| Cursor over page number | Stable when new posts arrive; cannot "jump to page 7". |
| `useReducer` over several `useState`s | Slightly more code, but transitions are pure, testable and impossible states are avoided. |
| No virtualization | Simple and fine for hundreds of posts; DOM grows unbounded in a very long session. |
| Auto-load only (no "Load more" button) | Smooth UX, but the footer is unreachable and some users prefer explicit control. |
| `rootMargin: 300px` | Prefetches early so users rarely see the skeleton; may load a page the user never scrolls to. |

### What to add with more time

- **Virtualization** ( `react-window` / `@tanstack/react-virtual` ) with dynamic row heights for 10k+ posts.
- **Scroll restoration**: cache `items` + `cursor` (React Query `useInfiniteQuery` or a store) and restore `scrollY` on back navigation.
- **New posts banner / pull to refresh**: poll for posts newer than `items[0].id`, show "5 new posts", prepend on click.
- **Images**: `loading="lazy"`, fixed aspect-ratio boxes to avoid layout shift.
- **Retry with exponential backoff** for transient failures; offline detection ( `navigator.onLine` ).
- **Tests**: reducer unit tests; React Testing Library with a mocked `IntersectionObserver` for "scroll → append", "error → retry", "end".

---

## 13. Counter questions the interviewer may ask

### Requirements

**Q1. Why did you pick cursor-based pagination instead of `?page=N`?**

Offsets shift when posts are inserted or deleted at the top, so you get duplicates or skipped posts while scrolling a live feed. A cursor ("older than id X") is stable. Trade-off: no random page jumps, which a feed doesn't need anyway.

**Q2. What if the API only supports offset pagination?**

Use `offset = items.length` and keep the reducer's de-duplication by id – it hides duplicates caused by new posts. Skipped items can't be fully prevented; you'd ask the backend for a `since` / `before` timestamp parameter.

### Q3. Should there also be a "Load more" button?

Pure auto-load makes the footer unreachable and is harder for some assistive-tech users. A common compromise: auto-load the first N pages, then show a "Load more" button. Our Retry button already uses the same `loadMore`, so adding it is one line.

## State design

### Q4. Why is `inFlight` a ref and not state?

State updates are asynchronous; two observer callbacks in the same tick both read the old `status`. A ref is read and written synchronously and doesn't cause a re-render, so it's a true lock.

### Q5. Why one `status` field instead of `isLoading` and `isError`?

Two booleans allow the impossible combination "loading and error". A single enum makes the UI a function of one value and the transitions explicit in the reducer.

### Q6. Why not store `isEmpty` / `isEnd` in state?

They are derived from `items.length` and `hasMore`. Storing them duplicates the truth and risks them getting out of sync.

## Hard challenges

### Q7. Why IntersectionObserver instead of a scroll listener?

A scroll listener fires dozens of times per second on the main thread and needs throttling plus `getBoundingClientRect` math (layout reads). The observer is async, browser-optimized and gives exactly the "is it near the viewport" signal we need.

### Q8. What is the stale closure problem here?

The observer callback is created once. If it called `loadMore` directly, it would use the `loadMore` from the first render, whose cursor is `null`, so every scroll would fetch page 1. Storing the latest callback in a ref fixes it without re-creating the observer on every render.

### Q9. The first page doesn't fill a 4K monitor. What happens?

The sentinel stays visible, so the observer never sees a *change*. We disable the observer while loading and re-create it afterwards; a new observer reports the current intersection immediately, so the next page loads until the screen is full.

### Q10. How do you avoid hammering a failing server?

On error the observer is disabled (`status !== 'idle'`), so only a user click on Retry triggers the next request. With more time: exponential backoff with jitter and a max retry count.

### Q11. What happens in React StrictMode in development?

Effects run mount → cleanup → mount, so the observer is created twice. The `inFlight` lock and the reducer's dedupe ensure only one copy of each post; the abort-on-unmount cleanup runs when nothing is in flight yet.

## Implementation steps

### Q12. Why `key={post.id}` and not the index?

Indexes are stable only when you append. Once you prepend new posts or remove one, index keys make React reuse the wrong DOM nodes, losing focus and component state.

### Q13. Why wrap `PostCard` in `memo`?

Appending page 6 re-renders `Feed`. Without `memo`, all 50 existing cards re-render too. With `memo` and stable `post` objects (we never recreate old items), only the 10 new cards render.

### Q14. Why `rootMargin: '300px'`?

It starts loading before the user reaches the bottom, hiding most of the network latency. Tune it to roughly one screen of content; too large wastes requests, too small shows the skeleton every time.

## Performance and "what if" twists

### Q15. What if the user scrolls through 1 million posts?

The DOM would hold a million nodes and memory would grow unbounded. Use **virtualization** (render only visible rows with dynamic heights) and optionally drop very old pages from memory and re-fetch them when scrolling back up.

### Q16. What if the API is slow (5 s per page)?

Increase the prefetch margin, keep the skeleton, and consider prefetching page N+1 as soon as page N arrives. Add a timeout with `AbortController` so a hung request turns into a retryable error.

### Q17. What if the user navigates to a post and presses Back?

Our state lives in the component, so it's lost and the feed restarts at the top. Fix: lift the data into a cache (React Query `useInfiniteQuery`, context or a store), save `scrollY` in `sessionStorage` or history state, and restore it after the cached items render.

### Q18. What if new posts are published while the user is reading?

Don't insert them at the top automatically – that shifts the content under the user's eyes. Poll (or use WebSocket) for posts newer than `items[0].id` and show a "N new posts" pill that prepends them and scrolls to top on click.

### Q19. How would posts with images change the design?

Use `loading="lazy"` and `decoding="async"`, and reserve space with `aspect-ratio` / width + height so images loading in don't shift the layout (CLS) and don't move the sentinel around.

## Trade-offs

### Q20. Why not just use React Query's `useInfiniteQuery`?

In a real app I would: it gives caching, dedupe, retries and back-navigation for free. In an interview, building it by hand shows I understand the race conditions it solves; I'd mention it as the production choice.

### Q21. How would you test this?

Unit test the pure `feedReducer` (append, dedupe, error keeps items). Component tests with React Testing Library and a mocked `IntersectionObserver` that I trigger manually: assert page 2 appends, error shows Retry, Retry recovers, end message appears. An E2E (Playwright) test scrolls the real page.

### Q22. Is this accessible?

`role="feed"` with `aria-busy` tells screen readers when content is being added; each post is a focusable `<article>` with `aria-posinset` and `aria-setsize="-1"` (unknown total); errors use `role="alert"`; the shimmer respects `prefers-reduced-motion`.